

application used in the program is:

```
If mywordapp Is Nothing Then
Set mywordapp = CreateObject("Word.
application")
mywordapp.Visible = False
End If
```

The necessary program code is almost associated with click events of menu items. You can create a new document and activate it with the following code (which is used in 'filenew' click events):

```
If mywordapp Is Nothing Then
Set mywordapp = CreateObject("Word.
application")
Set thisdoc = mywordapp.Documents.Add
thisdoc.Activate
```

Open an existing document using Microsoft Windows common dialogue control and activate it using the following code (which is used in 'fileopen' click events):

```
CommonDialog1.Filter = "Word
Document|*.doc"
CommonDialog1.ShowOpen
filename = CommonDialog1.filename
Set thisdoc = mywordapp.Documents.
Open(filename)
thisdoc.Activate
```

The code used in 'filesave' click event saves the document:

```
Private Sub filesave_Click(Index As
Integer)
Set thisdoc = mywordapp.
ActiveDocument
If thisdoc.Saved = True Then
vbresult = MsgBox("File already
saved, overwrite it?", vbQuestion,
"FILE SAVE")
If vbresult = True Then
thisdoc.Save
Else
thisdoc.Close (wdDoNotSaveChanges)
End If
Else
thisdoc.Save
End If
End Sub
```

The code used in 'filesaves' click event opens common dialogue controls, asks for the file name to save and saves the document with that file name:

```
Private Sub filesaves_Click(Index As
```

```
Integer)
Dim filename As String
CommonDialog1.ShowSave
filename = CommonDialog1.filename
Set thisdoc = mywordapp.
ActiveDocument
thisdoc.SaveAs filename, , , , True
thisdoc.Close (wdDoNotSaveChanges)
mywordapp.Quit (wdDoNotSaveChanges)
End Sub
```

The code used in 'fileclose' click event closes the program:

```
Private Sub fileclose_Click(Index As
Integer)
If thisdoc Is Nothing Then
mywordapp.Quit (wdDoNotSaveChanges)
resetcombar ' to enable disabled
menu bar items
End
Else
resetcombar ' to enable disabled
menu bar items
mywordapp.Documents(1).ActiveWindow.
WindowState = wdWindowStateNormal
mywordapp.Documents(1).Close
(wdDoNotSaveChanges)
mywordapp.Quit
(wdPromptToSaveChanges)
End
End If
End Sub
```

You can set and reset a few menu bar items of Microsoft Word in the OLE environment by using two user-defined procedures:

```
Private Sub resetcombar()
If thisdoc Is Nothing Then
Exit Sub
Else
Application.CommandBars.ActiveMenu
Bar.Controls.Item(3).Enabled = True
Application.CommandBars.ActiveMenu
Bar.Controls.Item(6).Enabled = True
End If
End Sub
Private Sub setcombar()
If OLE1.AppIsRunning = True Then
mywordapp.CommandBars.ActiveMenuBar.
Enabled = True
Application.CommandBars.ActiveMenu
Bar.Controls.Item(3).Enabled = False
Application.CommandBars.ActiveMenu
Bar.Controls.Item(6).Enabled = False
End If
```

End Sub

The view is set using 'myview' and zoomed to fit the pages in the active window using the code below:

```
Set myview = thisdoc.ActiveWindow.
View
myview.Zoom.Percentage = 75 'for
single page
myview.Zoom.Percentage = 42 'for
many pages
```

Either the gotopg1_Click event or the PAGE1_Click event transfers the page number related information to get the page contents onto the OLE and call the procedure gotoppgen():

```
Private Sub gotopg1_Click(Index As
Integer)
pgindex = Index
gotoppgen
Exit Sub
End Sub
Private Sub PAGE1_Click(Index As
Integer)
.
.
Select Case Index
Case 0:
pgindex = 2211
Case 1:
pgindex = 2221
Case 2:
pgindex = 2231
.
.
gotoppgen
End Sub
```

The code in gotoppgen() calculates the page number, copies the page contents, pastes onto the OLE, then activates Word on the OLE.

The code associated with 'paste back' command button click event pastes the text on the OLE back into the Word document. The line of code used for this purpose is 'thisrange.PasteAndFormat wdPasteDefault'

The OLE1.doverb(1) or OLE1.doverb(vbOLEShow) used at several places in the code opens the object for editing and activates the application that created the document in the OLE. The program code handles errors and displays system error message. The